

Projet SI-SOR – Mars 2026

Développement d'une application de gestion Kanban

Évaluation

Démonstration à effectuer

Rapport d'une page à réaliser

(liste des technologies utilisées et fonctionnalités mises en œuvre)

Sources à déposer sur Moodle au plus tard le 20/03/2026

Le travail peut être fait en binôme

1. Présentation du cadre applicatif

On souhaite créer une application web de gestion de tâches basée sur la méthode **Kanban**.

Le principe est de permettre à des utilisateurs d'organiser leur travail sous forme de **tableaux** contenant des **colonnes** et des **tâches**. Chaque tâche peut être déplacée entre différentes colonnes afin de représenter son état d'avancement (par exemple : *À faire*, *En cours*, *Terminé*).

Un utilisateur peut créer plusieurs tableaux, inviter d'autres utilisateurs à collaborer, créer des tâches et suivre l'évolution du travail.

Chaque tâche peut contenir une description détaillée, être assignée à un utilisateur, posséder une date limite et recevoir des commentaires.

Le but du projet est de construire une application *n*-tiers qui utilise des éléments étudiés dans le cadre des UE SOR et SI : client React ou Vue, reverse proxy nginx, serveur Deno/Oak, authentification avec JWT, serveur Tomcat/Servlet/Spring/JPA, bases de données SQL/MongoDB.

L'architecture demandée est illustrée dans la figure 1. Le développement de l'application sera versionné avec **Git**.

2. Fonctionnalités à mettre en œuvre

Les fonctionnalités soulignées sont obligatoires.

– Partie publique :

- Création de compte utilisateur
- Authentification
- Consultation des tableaux auxquels l'utilisateur participe
- Création d'un tableau
- Invitation d'utilisateurs à un tableau

– Gestion des tableaux :

- Création, modification et suppression de tableaux

- Gestion des membres d'un tableau
- Création et suppression de colonnes
- Gestion des tâches
 - Création, modification et suppression de tâches
 - Attribution d'une tâche à un utilisateur
 - Ajout d'une date limite et d'une priorité
 - Ajout de commentaires
- Partie administration :
 - Gestion des comptes utilisateurs
 - Statistiques globales :
 - nombre de tableaux
 - nombre de tâches
 - activité des utilisateurs
- Options :
 - IHM :
 - Réorganisation des colonnes
 - Déplacement de tâches entre colonnes (drag & drop)
 - Ajout de pièces jointes
 - Temps réel :
 - Mise à jour instantanée des tableaux avec WebSockets (par exemple lorsqu'une tâche est déplacée)
 - Notification lorsqu'une tâche est assignée ou commentée
 - Historique :
 - Journal des actions (création, déplacement et modification des tâches)
 - Recherche :
 - Recherche de tâches par mot-clé, utilisateur ou priorité

3. Technologies à utiliser

- Client (*frontend*) : **React** ou **Vue**
- Reverse proxy (*optionnel*) : **nginx** avec TLS pour HTTPS
- Serveur n°1 (*backend*) : **Deno** (Oak, WebSocket, JWT), authentification des utilisateurs
- Serveur n°2 (*backend*) : **Tomcat** (Servlet, Spring, JPA), accès aux bases de données et gestion du modèle
- Bases de données : **SQL** (MariaDB ou SQLite) et **MongoDB**

4. Architecture à mettre en œuvre

Le client React ou Vue communique en HTTP avec le serveur Deno. nginx est intégré en fin de projet, en option, pour chiffrer les communications (HTTPS).

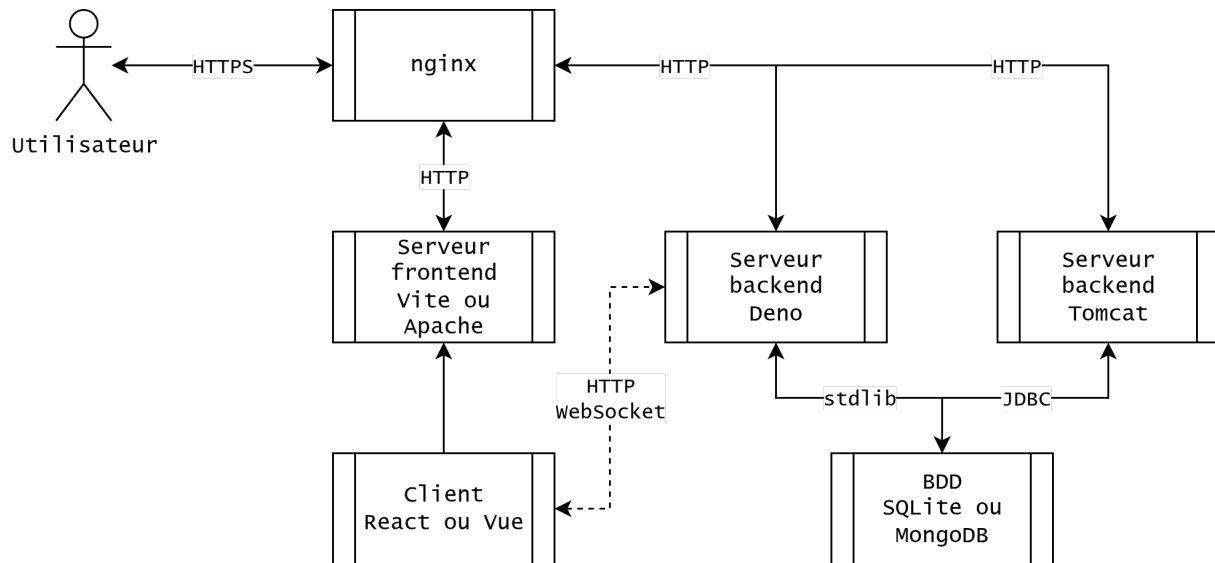


FIGURE 1 : Architecture demandée pour l'application

Le serveur Deno est chargé de sécuriser l'application web. Il vérifie que les requêtes reçues sont autorisées (c'est-à-dire que le client est authentifié). Il peut accéder directement à la base de données pour effectuer cette vérification. Un fois la vérification faite, il retransmet le cas échéant la requête au serveur Tomcat, ou indique au client que l'opération est impossible.

Le serveur Tomcat gère tous les autres accès aux bases de données via des API REST. On peut utiliser les Servlets et/ou Spring.

Les commentaires avec les pièces jointes éventuelles sont à stocker dans une base MongoDB. Le reste des données de l'application sont stockées dans une base SQL.

5. Organisation et évaluation

- Le travail peut être effectué en binôme ;
- L'archive finale doit être déposée sur le cours Moodle « *Projet SI-SOR (M1 - TIIL)* » et contenir :
 - Un résumé d'environ une page donnant une liste des fonctionnalités mises en œuvre et les technologies utilisées ;
 - Le schéma de la base de données SQL et la structure des collections MongoDB ;
 - La liste des API et en quelques mots ce que fait chaque API ;
 - Les sources de l'application ;
- L'évaluation se fera sur la base de ce dépôt et surtout d'une **démonstration à la fin du projet**.

6. Critères de notation

- La fiabilité de l'application : la **gestion des erreurs** doit être traitée avec soin (cas des formulaires mal remplis, remontée des erreurs d'une couche applicative à l'autre, etc.) ;
- Le niveau fonctionnel atteint ;
- La qualité et l'exhaustivité des jeux de tests ;

- La qualité du code (organisation, concision, formatage).